MargDarshak-AI

# Low-Level Design

Schema, API, analysis contract and implementation details

| | |
|---|---|
| Status | Draft v1.0 |
| Date | 7 November 2026 |
| Note | Table, route and file names are the proposed design. Rename them to match the actual code where they differ. |

## 1. Project layout

```
margdarshak-ai/
  client/                      React app
    src/
      pages/                   Home, Assessment, Dashboard, Saved, Login, Signup
      components/              CareerCard, Roadmap, ProjectList, ProgressBar, Disclaimer
      api/client.js            fetch wrapper (credentials, error mapping)
      state/AuthContext.jsx    current user, sign-in, sign-out
  server/                      Node + Express API
    src/
      routes/                  auth.js, assessments.js, progress.js, feedback.js
      services/                gemini.js, prompt.js, analysisSchema.js
      middleware/              requireAuth.js, rateLimit.js, errorHandler.js
      db/                      pool.js, migrations/
      config.js                reads and checks environment variables
      app.js, server.js
```

Routes stay thin: parse, validate, call a service, respond. Anything involving the model lives in `services/` so it can be tested without HTTP.

## 2. Database schema (PostgreSQL)

```
users
  id            uuid  PK  default gen_random_uuid()
  email         text  UNIQUE NOT NULL        (stored lowercase)
  name          text  NOT NULL
  password_hash text  NULL                   (null for Google-only accounts)
  google_sub    text  UNIQUE NULL
  created_at    timestamptz default now()

assessments
  id            uuid  PK
  user_id       uuid  FK users ON DELETE CASCADE
  answers       jsonb NOT NULL               (interests, strengths, preferences)
  context       jsonb NULL                   (board, stream, course, location...)
  instructions  text  NULL
  career_count  int   CHECK (career_count BETWEEN 5 AND 10)
  created_at    timestamptz default now()

analyses
  id            uuid  PK
  assessment_id uuid  FK assessments ON DELETE CASCADE  UNIQUE
  result        jsonb NOT NULL               (validated model output)
  model         text  NOT NULL
  created_at    timestamptz default now()

progress
  id            uuid  PK
  user_id       uuid  FK users ON DELETE CASCADE
```

```
  analysis_id   uuid  FK analyses ON DELETE CASCADE
  item_key      text  NOT NULL               (e.g. "month2.task1")
  done          boolean default false
  updated_at    timestamptz default now()
  UNIQUE (user_id, analysis_id, item_key)

feedback
  id            uuid  PK
  user_id       uuid  FK users ON DELETE CASCADE
  analysis_id   uuid  FK analyses ON DELETE CASCADE
  rating        int   CHECK (rating BETWEEN 1 AND 5)
  comment       text  NULL
  created_at    timestamptz default now()

Indexes: assessments(user_id, created_at DESC), progress(user_id, analysis_id)
```

Ownership checks join back to `user_id`: an analysis is reachable only through an assessment that belongs to the signed-in user.

## 3. API

| Method and path | Auth | What it does |
|---|---|---|
| POST /api/auth/signup | No | Create account (name, email, password). Sets session cookie. |
| POST /api/auth/login | No | Email and password sign-in. Sets session cookie. |
| POST /api/auth/google | No | Verify Google ID token, find or create user. Returns 404 if Google is not configured. |
| POST /api/auth/logout | Yes | Clears the cookie. |
| GET /api/auth/me | Yes | Current user, or 401. |
| GET /api/config | No | Public flags, such as whether Google sign-in is enabled. |
| POST /api/assessments | Yes | Save answers, call Gemini, store and return the analysis. |
| GET /api/assessments | Yes | List the user's saved assessments (id, date, top career names). |
| GET /api/assessments/:id | Yes | One assessment with its analysis and progress. |
| DELETE /api/assessments/:id | Yes | Delete it and everything under it. |
| PUT /api/progress | Yes | Set one roadmap item done or not done. |
| POST /api/feedback | Yes | Rating and optional comment for an analysis. |

### Request and response for POST /api/assessments

```
Request
{
  "answers":      { "interests": ["design","maths"], "strengths": ["..."], "preferences": {...} },
  "context":      { "board": "CBSE", "stream": "Science", "state": "Haryana" },   // optional
  "instructions": "I would rather not do a desk job",                              // optional
  "careerCount":  5                                                                // 5 to 10
}

Response 201
{ "assessmentId": "uuid", "analysis": { ...see section 4... } }
```

## 4. Shape of the analysis

This is the contract between the model, the server and the dashboard. The server rejects anything that does not match.

```
{
  "summary": "string",
  "careers": [                          // length == careerCount
    {
      "title": "string",
      "whyItFits": "string",
      "skillsUsed": ["string"],
      "skillGaps": ["string"],
      "firstSteps": ["string"]          // 2 to 5 items
    }
  ],
  "roadmap": [                          // 6 entries, one per month
    { "month": 1, "focus": "string", "tasks": ["string"] }
  ],
  "projects": [
    { "title": "string", "description": "string", "level": "beginner" }
  ],
  "disclaimer": "string"
}
```

Progress keys are built from the roadmap, for example `m2.t1` for month 2, first task. Because the dashboard draws from this JSON, a new field in the schema needs a matching component, but a different set of careers needs no code change.

## 5. Calling Gemini

1. **prompt.js** builds the system instruction (role, tone, India context, JSON-only, the schema) and a user message from the answers.
2. Free-text fields are length-capped and wrapped in clear delimiters, so instructions typed by the student cannot override the system instruction.
3. **gemini.js** sends the request with the JSON response mode on and a fixed timeout.
4. The raw text is parsed, then checked with **analysisSchema.js**: career count matches, no empty strings, step and task counts within limits.
5. On failure it retries once, adding the validation error to the prompt. A second failure raises a typed error.
6. On success, the result is saved inside a transaction together with the assessment.

Model name and timeout come from config, so they can change without a code edit. Errors map to: timeout 504, bad output after retry 502, quota or rate limit 503 with a retry hint.

## 6. Validation rules

| Field | Rule |
|---|---|
| email | Valid format, lowercased, max 254 chars |
| password | 8 to 72 characters (bcrypt limit), at least one letter and one number |
| name | 1 to 80 characters, trimmed |
| careerCount | Integer 5 to 10, default 5 |
| answers | Required fields present; lists limited to 15 items; each text 200 chars max |
| instructions | Max 500 characters |

| Field | Rule |
|---|---|
| comment | Max 1000 characters |

## 7. Sessions and access control

- On sign-in the server signs a token with the user id and a 7-day expiry, and sets it as an HTTP-only, SameSite cookie (Secure in production).
- `requireAuth` verifies the cookie, loads the user id and puts it on the request. Missing or expired: 401.
- Every query uses that id: `WHERE user_id = $1`. A resource that exists but belongs to someone else returns 404, not 403.
- Google flow: client sends the ID token, server verifies signature, audience and expiry, then matches on `google_sub`, falling back to email to link an existing account.
- Rate limits: 10 sign-in attempts per 15 minutes per IP; 5 analysis requests per hour per user.
- Also on: Helmet headers, CORS limited to the known origin, JSON body size limit.

## 8. Frontend

| Route | Page | Notes |
|---|---|---|
| / | Home | Short pitch, start button, disclaimer |
| /login, /signup | Auth | Google button shown only if /api/config says it is enabled |
| /assessment | Assessment | Multi-step form; keeps answers in memory so a failed call can be retried |
| /dashboard/:id | Dashboard | Career cards, roadmap, projects, progress bar; all driven by the analysis JSON |
| /saved | Saved | List of past assessments with delete |

- Auth state lives in one context. On sign-out it is cleared and any cached assessment data is dropped, so the next person on the same device sees nothing.
- Progress ticks update the screen immediately and roll back with a message if the request fails.
- Loading, empty and error states exist for every data screen; the analysis wait shows steps, not a bare spinner.

## 9. Errors the user can see

| Situation | Status | Message shown |
|---|---|---|
| Invalid form data | 400 | Highlights the field and says what to fix |
| Not signed in or session expired | 401 | Redirect to sign-in, keep the typed answers |
| Wrong email or password | 401 | Same message for both cases |
| Email already registered | 409 | Suggest signing in |
| Too many requests | 429 | Tell them how long to wait |
| Model timeout or bad output | 502 / 504 | Could not build your plan; answers are kept, try again |
| Anything else | 500 | Generic message; details only in server logs |

## 10. Testing

- **Unit:** schema validation with good and bad model output, prompt builder, password and token helpers.
- **API:** each route against a test database, including the case where user B asks for user A's assessment.
- **Model service:** mocked Gemini returning valid, malformed, empty and slow replies to check retry and timeout.
- **Frontend:** dashboard renders with 5 and with 10 careers; form validation; sign-out clears state.
- **Manual before release:** run a handful of realistic student profiles and read the output for quality and tone.